



LOVELY
PROFESSIONAL
UNIVERSITY

SUMMER TRAINING/INTERNSHIP PROJECT REPORT

Project name: **Electric Grid Failure Prediction**

Submitted by : Vemireddy Venkataramireddy.

Registration Number: 12307726

Course Code : PETV50

Under the Guidance of:

Dr. Nishant Agnihotri and Puneet Kumar

School of Computer Science and Engineering

Certificate :

			CENTRE FOR PROFESSIONAL ENHANCEMENT	Certificate No. 409532
				
Certificate of Merit				
This is to certify that Mr./Ms. <u>Vemireddy Venkataramireddy</u> S/D/W/o <u>Mr. Vemireddy Brahmananda Reddy</u>				
student of <u>School of Computer Science and Engineering</u> Registration No. <u>12307726</u>				
pursuing <u>Bachelor of Technology (Computer Science and Engineering)</u> completed				
skill development course named <u>Basics of DSA using C++</u>				
organized by <u>Centre for Professional Enhancement</u> Lovely Professional University				
from <u>10 June 2025</u> to <u>23 July 2025</u> and obtained <u>A</u> Grade.				
				
Date of Issue : 13-08-2025 Place of Issue: Phagwara (India)	Prepared by (Administrative Officer-Records)	Programme Coordinator Centre for Professional Enhancement	Head of School School of Computer Science and Engineering	

ACKNOWLEDGEMENT:

I express my sincere gratitude to my faculty mentor, **Dr. Nishant Agnihotri**, for his invaluable guidance, support, and encouragement throughout the internship period. His mentorship played a vital role in shaping my understanding and approach to the project.

I would also like to thank **Mr. Puneet Kumar** for his technical assistance and insightful suggestions, which helped me overcome challenges during the course of the project.

My heartfelt thanks to the **School of Computer Science and Engineering** for providing me with the opportunity to undertake this internship and supporting my academic and professional development.

During the internship, I worked on a project titled "**Electric Grid Failure Prediction**." This project enabled me to apply theoretical knowledge in a practical environment and enhanced my skills in data structures, algorithms, and real-time data analysis. It also helped me develop a deeper understanding of predictive modeling and its applications in critical infrastructure.

This internship has been a valuable learning experience and has significantly contributed to my growth as a student and an aspiring engineer. I am truly thankful to everyone who supported me throughout this journey.

TABLE OF CONTENTS

1. Introduction
2. Project Details
3. Implementation
4. Results and Discussion
5. Conclusion

CHAPTER 1: INTRODUCTION:

Overview of Training Domain:

The domain of this training primarily revolves around **java** and the **fundamentals of Data Structures**, with a particular focus on applying these concepts in the simulation of real-time systems. The training involved an in-depth understanding of core java principles along with practical implementation of key data structures such as **Graphs, Queues, and Priority Queues**. The use of java enabled efficient memory management and faster execution, which are essential for real-time system simulations. By integrating data structures with algorithmic strategies, this training aimed to develop a strong foundation for solving complex computational problems in dynamic environments like electric grids.

Objective of the Project:

The main objective of the project is to **design and implement a simulation system capable of predicting potential electric grid failures** using **java**. The simulation leverages fundamental data structures, including **Graphs** for modeling the network of the grid, **Queues** for managing event-driven processes, and **Priority Queues** to handle failure probabilities and scheduling. The project seeks to analyze various grid parameters such as voltage, current, and temperature to calculate failure probabilities in different nodes of the grid. Through this simulation, the aim is to enhance proactive monitoring and preventive measures in grid management systems.

CHAPTER 2: PROJECT DETAILS:

Title of the Project : “Electric Grid Failure Prediction”

Problem Definition:

Power grids are prone to failures due to overloads, temperature rise, or voltage/current fluctuations. The goal is to predict these failures using a software model.

Scope and Objectives:

- Identify high-risk grid nodes
- Predict cascading failures
- Find critical paths from failure nodes to generators
- Incorporate weather influence in grid stability

System Requirements:

- JDK 8 or higher
- 4GB RAM minimum
- Command-line environment or VS Code

Code:

```
J GridFailurePredictor.java > ...
1  import java.util.*;
2  import java.time.*;
3
4  /**
5   * Electric Grid Failure Predictor
6   * Uses DSA concepts: Graph, BFS, PriorityQueue, Dijkstra-like path.
7   */
8
9  class GridNode {
10     String nodeId;
11     double voltage;
12     double current;
13     double temperature;
14     double failureProbability;
15     List<GridNode> neighbors;
16
17     public GridNode(String nodeId) {
18         this.nodeId = nodeId;
19         this.neighbors = new ArrayList<>();
20     }
21
22     public void updateReadings(double voltage, double current, double temperature) {
23         this.voltage = voltage;
24         this.current = current;
25         this.temperature = temperature;
26         calculateFailureProbability();
27     }
28
29     private void calculateFailureProbability() {
30         double voltageScore = Math.abs(voltage - 220) / 50;
31         double currentScore = current > 15 ? (current - 15) / 10 : 0;
32         double tempScore = temperature > 80 ? (temperature - 80) / 20 : 0;
33
34         this.failureProbability = Math.min(0.3 * voltageScore + 0.4 * currentScore + 0.3 * tempScore, 1.0);
35     }
36
37     public void addNeighbor(GridNode node) {
```

```

36
37     public void addNeighbor(GridNode node) {
38         neighbors.add(node);
39     }
40
41     @Override
42     public String toString() {
43         return String.format(
44             format:"%s [voltage: %.1f V, Current: %.1f A, Temperature: %.1f °C, Failure Probability: %.1f%%]",
45             nodeId, voltage, current, temperature, failureProbability * 100
46         );
47     }
48 }
49
50 class ElectricGrid {
51     private Map<String, GridNode> nodes;
52
53     public ElectricGrid() {
54         nodes = new HashMap<>();
55     }
56
57     public void addNode(GridNode node) {
58         nodes.put(node.nodeId, node);
59     }
60
61     public void connectNodes(String nodeId1, String nodeId2) {
62         GridNode node1 = getNode(nodeId1);
63         GridNode node2 = getNode(nodeId2);
64         if (node1 != null && node2 != null) {
65             node1.addNeighbor(node2);
66             node2.addNeighbor(node1);
67         }
68     }
69
70     public List<GridNode> getPotentialCascade(String startNodeId) {
71         List<GridNode> cascade = new ArrayList<>();

```

```

70     public List<GridNode> getPotentialCascade(String startNodeId) {
71         List<GridNode> cascade = new ArrayList<>();
72         Set<String> visited = new HashSet<>();
73         Queue<GridNode> queue = new LinkedList<>();
74
75         GridNode startNode = getNode(startNodeId);
76         if (startNode == null) return cascade;
77
78         queue.add(startNode);
79         visited.add(startNodeId);
80
81         while (!queue.isEmpty()) {
82             GridNode current = queue.poll();
83             cascade.add(current);
84
85             for (GridNode neighbor : current.neighbors) {
86                 if (!visited.contains(neighbor.nodeId)) {
87                     visited.add(neighbor.nodeId);
88                     queue.add(neighbor);
89                 }
90             }
91         }
92
93         return cascade;
94     }
95
96     public List<String> findCriticalPath(String startNodeId, String endNodeId) {
97         Map<String, Double> distances = new HashMap<>();
98         Map<String, String> previous = new HashMap<>();
99         PriorityQueue<String> queue = new PriorityQueue<>(Comparator.comparingDouble(distances::get));
100
101         for (String nodeId : nodes.keySet()) {
102             distances.put(nodeId, Double.MAX_VALUE);
103         }
104         distances.put(startNodeId, value:0.0);

```

```

104         distances.put(startNodeId, value:0.0);
105         queue.add(startNodeId);
106
107         while (!queue.isEmpty()) {
108             String current = queue.poll();
109             if (current.equals(endNodeId)) break;
110
111             for (GridNode neighbor : getNode(current).neighbors) {
112                 double alt = distances.get(current) + neighbor.failureProbability;
113                 if (alt < distances.get(neighbor.nodeId)) {
114                     distances.put(neighbor.nodeId, alt);
115                     previous.put(neighbor.nodeId, current);
116                     queue.add(neighbor.nodeId);
117                 }
118             }
119         }
120
121         List<String> path = new ArrayList<>();
122         for (String at = endNodeId; at != null; at = previous.get(at)) {
123             path.add(at);
124         }
125         Collections.reverse(path);
126
127         return path.size() > 1 && path.get(index:0).equals(startNodeId) ? path : Collections.emptyList();
128     }
129
130     public List<GridNode> getHighRiskNodes() {
131         List<GridNode> risky = new ArrayList<>();
132         for (GridNode node : nodes.values()) {
133             if (node.failureProbability > 0.3) {
134                 risky.add(node);
135             }
136         }
137         risky.sort((a, b) -> Double.compare(b.failureProbability, a.failureProbability));
138         return risky;

```



```

139     }
140
141     public GridNode getNode(String nodeId) {
142         return nodes.get(nodeId);
143     }
144
145     public Collection<GridNode> getAllNodes() {
146         return nodes.values();
147     }
148 }
149
150 class WeatherData {
151     LocalDateTime timestamp;
152     double temperature;
153     double windSpeed;
154     double precipitation;
155
156     public WeatherData(double temperature, double windSpeed, double precipitation) {
157         this.timestamp = LocalDateTime.now();
158         this.temperature = temperature;
159         this.windSpeed = windSpeed;
160         this.precipitation = precipitation;
161     }
162 }
163
164 public class GridFailurePredictor {
165     private ElectricGrid grid;
166     private Queue<WeatherData> weatherHistory;
167     private final int WEATHER_HISTORY_SIZE = 100;
168     private Scanner scanner;
169
170     public GridFailurePredictor() {
171         grid = new ElectricGrid();
172         weatherHistory = new LinkedList<>();
173         scanner = new Scanner(System.in);
174
175         scanner = new Scanner(System.in);
176         initializeSampleGrid();
177     }
178
179     private void initializeSampleGrid() {
180         String[] nodeIds = {"SUB1", "SUB2", "SUB3", "TR1", "TR2", "TR3", "GEN1", "GEN2"};
181         for (String id : nodeIds) {
182             GridNode node = new GridNode(id);
183             node.updateReadings(
184                 200 + Math.random() * 40,
185                 5 + Math.random() * 15,
186                 30 + Math.random() * 60
187             );
188             grid.addNode(node);
189
190             grid.connectNodes(nodeId1: "GEN1", nodeId2: "SUB1");
191             grid.connectNodes(nodeId1: "GEN2", nodeId2: "SUB2");
192             grid.connectNodes(nodeId1: "SUB1", nodeId2: "TR1");
193             grid.connectNodes(nodeId1: "SUB1", nodeId2: "TR2");
194             grid.connectNodes(nodeId1: "SUB2", nodeId2: "TR2");
195             grid.connectNodes(nodeId1: "SUB2", nodeId2: "TR3");
196             grid.connectNodes(nodeId1: "SUB3", nodeId2: "TR1");
197             grid.connectNodes(nodeId1: "SUB3", nodeId2: "TR3");
198         }
199
200     public void addWeatherData(WeatherData data) {
201         weatherHistory.offer(data);
202         if (weatherHistory.size() > WEATHER_HISTORY_SIZE) {
203             weatherHistory.poll();
204         }
205         adjustGridParametersBasedOnWeather();
206     }
207
208     private void adjustGridParametersBasedOnWeather() {
209         WeatherData currentWeather = weatherHistory.peek();
210         if (currentWeather == null) return;
211
212         for (GridNode node : grid.getAllNodes()) {
213             node.updateReadings(node.voltage, node.current, node.temperature + currentWeather.temperature * 0.1);
214         }
215     }
216
217     public void predictFailures() {
218         System.out.println(x:"n=== Grid Status Report ===");
219         for (GridNode node : grid.getAllNodes()) {
220             System.out.println(node);
221         }
222
223         List<GridNode> highRiskNodes = grid.getHighRiskNodes();
224         if (highRiskNodes.isEmpty()) {
225             System.out.println(x:"nAll nodes are stable.");
226         } else {
227             System.out.println(x:"nHigh Risk Nodes Detected:");
228             for (GridNode node : highRiskNodes) {
229                 System.out.println(">> " + node);
230                 if (node.failureProbability > 0.7) {
231                     List<GridNode> cascade = grid.getPotentialCascade(node.nodeId);
232                     System.out.println("    Possible cascade affecting " + cascade.size() + " nodes: " +
233                         cascade.stream().map(n -> n.nodeId).toList());
234                     List<String> pathToGen1 = grid.findCriticalPath(node.nodeId, endNodeId: "GEN1");
235                     List<String> pathToGen2 = grid.findCriticalPath(node.nodeId, endNodeId: "GEN2");
236
237                     System.out.println("    Critical path to GEN1: " + (pathToGen1.isEmpty() ? "No path" : pathToGen1));
238                     System.out.println("    Critical path to GEN2: " + (pathToGen2.isEmpty() ? "No path" : pathToGen2));
239                 }
240             }
241         }
242     }

```

```

244 private void displayMenu() {
245     System.out.println(x:"\n===== Electric Grid Failure Prediction =====");
246     System.out.println(x:"1. Enter Weather Data");
247     System.out.println(x:"2. Update Node Parameters");
248     System.out.println(x:"3. View All Nodes");
249     System.out.println(x:"4. Predict Failures");
250     System.out.println(x:"5. Exit");
251     System.out.print(s:"Choose an option: ");
252 }
253
254 private void enterWeatherData() {
255     System.out.println(x:"\nEnter Current Weather Conditions:");
256     System.out.print(s:"Temperature (°C): ");
257     double temp = scanner.nextDouble();
258     System.out.print(s:"Wind Speed (km/h): ");
259     double wind = scanner.nextDouble();
260     System.out.print(s:"Precipitation (mm): ");
261     double precip = scanner.nextDouble();
262
263     addWeatherData(new WeatherData(temp, wind, precip));
264     System.out.println(x:"Weather data recorded.");
265 }
266
267 private void updateNodeParameters() {
268     System.out.println(x:"\nAvailable Nodes:");
269     for (GridNode node : grid.getAllNodes()) {
270         System.out.println(node.nodeId);
271     }
272
273     System.out.print(s:"\nEnter Node ID to update: ");
274     String nodeId = scanner.next();
275     GridNode node = grid.getNode(nodeId);
276     if (node == null) {
277         System.out.println(x:"Node not found!");
278         return;
279     }

```

```

281     System.out.println("\nCurrent: " + node);
282     System.out.print(s:"New Voltage: ");
283     double voltage = scanner.nextDouble();
284     System.out.print(s:"New Current: ");
285     double current = scanner.nextDouble();
286     System.out.print(s:"New Temperature: ");
287     double temp = scanner.nextDouble();
288
289     node.updateReadings(voltage, current, temp);
290     System.out.println(x:"Node updated.");
291 }
292
293 private void viewAllNodes() {
294     System.out.println(x:"\n=== Current Grid Status ===");
295     for (GridNode node : grid.getAllNodes()) {
296         System.out.println(node);
297     }
298 }
299
300 public void run() {
301     boolean running = true;
302     while (running) {
303         displayMenu();
304         int choice = scanner.nextInt();
305         switch (choice) {
306             case 1 -> enterWeatherData();
307             case 2 -> updateNodeParameters();
308             case 3 -> viewAllNodes();
309             case 4 -> predictFailures();
310             case 5 -> {
311                 running = false;
312                 System.out.println(x:"Exiting system.");
313             }
314             default -> System.out.println(x:"Invalid choice! Please try again.");
315         }

```

```

311         running = false;
312         System.out.println(x:"Exiting system.");
313     }
314     default -> System.out.println(x:"Invalid choice! Please try again.");
315 }
316 scanner.close();
317 }
318
319 Run | Debug
320 public static void main(String[] args) {
321     GridFailurePredictor predictor = new GridFailurePredictor();
322     predictor.run();
323 }
324 }
325

```

Output:

```
===== Electric Grid Failure Prediction =====
```

1. Enter Weather Data
2. Update Node Parameters
3. View All Nodes
4. Predict Failures
5. Exit

Choose an option: 1

Enter Current Weather Conditions:

Temperature (°C): 30

Wind Speed (km/h): 23

Precipitation (mm): 12

Weather data recorded.

```
===== Electric Grid Failure Prediction =====
```

1. Enter Weather Data
2. Update Node Parameters
3. View All Nodes
4. Predict Failures
5. Exit

Choose an option: 2

Available Nodes:

GEN1

TR1

GEN2

SUB3

TR3

SUB2

TR2

SUB1

Enter Node ID to update: GEN1

Current: GEN1 [Voltage: 201.9 V, Current: 6.4 A, Temperature: 48.1 °C, Failure Probability: 10.9%]

New Voltage: 200

New Current: 12

New Temperature: 28

Node updated.

```
===== Electric Grid Failure Prediction =====
```

1. Enter Weather Data
2. Update Node Parameters
3. View All Nodes
4. Predict Failures
5. Exit

Choose an option: 3

=== Current Grid Status ===

GEN1 [Voltage: 200.0 V, Current: 12.0 A, Temperature: 28.0 °C, Failure Probability: 12.0%]

TR1 [Voltage: 238.3 V, Current: 15.1 A, Temperature: 82.5 °C, Failure Probability: 15.2%]

GEN2 [Voltage: 234.3 V, Current: 11.8 A, Temperature: 46.3 °C, Failure Probability: 8.6%]

SUB3 [Voltage: 232.8 V, Current: 19.9 A, Temperature: 69.3 °C, Failure Probability: 27.4%]

TR3 [Voltage: 230.9 V, Current: 15.0 A, Temperature: 74.7 °C, Failure Probability: 6.6%]

SUB2 [Voltage: 233.6 V, Current: 5.3 A, Temperature: 55.2 °C, Failure Probability: 8.1%]

TR2 [Voltage: 216.4 V, Current: 17.3 A, Temperature: 34.3 °C, Failure Probability: 11.3%]

SUB1 [Voltage: 216.6 V, Current: 15.7 A, Temperature: 85.5 °C, Failure Probability: 13.2%]

```
===== Electric Grid Failure Prediction =====
```

1. Enter Weather Data
2. Update Node Parameters
3. View All Nodes
4. Predict Failures
5. Exit

Choose an option: 4

=== Grid Status Report ===

GEN1 [Voltage: 200.0 V, Current: 12.0 A, Temperature: 28.0 °C, Failure Probability: 12.0%]

TR1 [Voltage: 238.3 V, Current: 15.1 A, Temperature: 82.5 °C, Failure Probability: 15.2%]

GEN2 [Voltage: 234.3 V, Current: 11.8 A, Temperature: 46.3 °C, Failure Probability: 8.6%]

SUB3 [Voltage: 232.8 V, Current: 19.9 A, Temperature: 69.3 °C, Failure Probability: 27.4%]

TR3 [Voltage: 230.9 V, Current: 15.0 A, Temperature: 74.7 °C, Failure Probability: 6.6%]

SUB2 [Voltage: 233.6 V, Current: 5.3 A, Temperature: 55.2 °C, Failure Probability: 8.1%]

TR2 [Voltage: 216.4 V, Current: 17.3 A, Temperature: 34.3 °C, Failure Probability: 11.3%]

SUB1 [Voltage: 216.6 V, Current: 15.7 A, Temperature: 85.5 °C, Failure Probability: 13.2%]

All nodes are stable.

CHAPTER 3: IMPLEMENTATION:

Tools Used:

- Visual Studio Code
- Java Runtime (JDK 8)

Methodology:

- Nodes modeled as objects
- Graph constructed using adjacency list
- PriorityQueue used for finding minimal failure paths
- Weather data factored into temperature calculate

Modules / Screenshots:

- Node Initialization
- Weather Data Input
- Failure Prediction Output

Code Snippets:

Java

```
public void updateReadings(double voltage, double current, double
temperature) {
    this.voltage = voltage;
    this.current = current;
    this.temperature = temperature;
    calculateFailureProbability();
}
```

CHAPTER 4: RESULTS AND DISCUSSION:

Output:

```
===== Electric Grid Failure Prediction =====
1. Enter Weather Data
2. Update Node Parameters
3. View All Nodes
4. Predict Failures
5. Exit
Choose an option: 1

Enter Current Weather Conditions:
Temperature (°C): 30
Wind Speed (km/h): 23
Precipitation (mm): 12
Weather data recorded.

===== Electric Grid Failure Prediction =====
1. Enter Weather Data
2. Update Node Parameters
3. View All Nodes
4. Predict Failures
5. Exit
Choose an option: 2

Available Nodes:
GEN1
TR1
GEN2
SUB3
TR3
SUB2
TR2
SUB1

Enter Node ID to update: GEN1

Current: GEN1 [Voltage: 201.9 V, Current: 6.4 A, Temperature: 48.1 °C, Failure Probability: 10.9%]
New Voltage: 200
New Current: 12
New Temperature: 28
Node updated.

===== Electric Grid Failure Prediction =====
1. Enter Weather Data
2. Update Node Parameters
3. View All Nodes
4. Predict Failures
5. Exit
Choose an option: 3

=== Current Grid Status ===
GEN1 [Voltage: 200.0 V, Current: 12.0 A, Temperature: 28.0 °C, Failure Probability: 12.0%]
TR1 [Voltage: 238.3 V, Current: 15.1 A, Temperature: 82.5 °C, Failure Probability: 15.2%]
GEN2 [Voltage: 234.3 V, Current: 11.8 A, Temperature: 46.3 °C, Failure Probability: 8.6%]
SUB3 [Voltage: 232.8 V, Current: 19.9 A, Temperature: 69.3 °C, Failure Probability: 27.4%]
TR3 [Voltage: 230.9 V, Current: 15.0 A, Temperature: 74.7 °C, Failure Probability: 6.6%]
SUB2 [Voltage: 233.6 V, Current: 5.3 A, Temperature: 55.2 °C, Failure Probability: 8.1%]
TR2 [Voltage: 216.4 V, Current: 17.3 A, Temperature: 34.3 °C, Failure Probability: 11.3%]
SUB1 [Voltage: 216.6 V, Current: 15.7 A, Temperature: 85.5 °C, Failure Probability: 13.2%]

===== Electric Grid Failure Prediction =====
1. Enter Weather Data
2. Update Node Parameters
3. View All Nodes
4. Predict Failures
5. Exit
Choose an option: 4

=== Grid Status Report ===
GEN1 [Voltage: 200.0 V, Current: 12.0 A, Temperature: 28.0 °C, Failure Probability: 12.0%]
TR1 [Voltage: 238.3 V, Current: 15.1 A, Temperature: 82.5 °C, Failure Probability: 15.2%]
GEN2 [Voltage: 234.3 V, Current: 11.8 A, Temperature: 46.3 °C, Failure Probability: 8.6%]
SUB3 [Voltage: 232.8 V, Current: 19.9 A, Temperature: 69.3 °C, Failure Probability: 27.4%]
TR3 [Voltage: 230.9 V, Current: 15.0 A, Temperature: 74.7 °C, Failure Probability: 6.6%]
SUB2 [Voltage: 233.6 V, Current: 5.3 A, Temperature: 55.2 °C, Failure Probability: 8.1%]
TR2 [Voltage: 216.4 V, Current: 17.3 A, Temperature: 34.3 °C, Failure Probability: 11.3%]
SUB1 [Voltage: 216.6 V, Current: 15.7 A, Temperature: 85.5 °C, Failure Probability: 13.2%]

All nodes are stable.
```

Output / Report:

The system prints a full grid report showing **voltage, current, temperature**, and **failure probability** for each node in the electric grid. Nodes exceeding **30% failure probability** are flagged as **high-risk**, helping operators prioritize immediate action. Additionally, the system calculates and displays **shortest or safest paths** from vulnerable nodes to **nearby power generators** using graph traversal algorithms such as **BFS** and **Dijkstra's Algorithm**. These paths help in rerouting power supply or isolating the faulty segments efficiently. The simulation updates dynamically, providing real-time feedback based on changing parameters. This enables better **decision-making** and enhances the reliability and robustness of grid operations.

Challenges Faced :

- Handling user input validation
- Ensuring correct graph traversal logic
- Integrating weather influence consistently

Learnings:

- Hands-on Java development
- Application of graph algorithms in real-world simulation
- Integration of dynamic data (weather) with static logic (grid)

CHAPTER 5: CONCLUSION:

This project simulates electric grid failure scenarios using **java programming** and fundamental **data structures**. It demonstrates how real-time **environmental** and **operational data**—such as voltage, current, and temperature—can be used to assess the health of grid components and predict potential failures. Each part of the grid is represented as a **node** in a **graph**, with key metrics being monitored continuously.

When a node's **failure probability** exceeds a threshold (30%), it is marked as **high risk**, helping prioritize timely maintenance or rerouting. The system also calculates **shortest paths** from critical nodes to backup generators using algorithms like **Dijkstra's**, improving response during emergencies.

By integrating **graphs, queues, and priority queues**, this project shows how predictive simulations can help prevent cascading failures, making the grid more robust and reliable. It highlights the real-world application of **java and data structures** in system-level problem solving.